

GUIA DE APRESENTACAO E DEFESA TECNICA DO CODIGO

SERNIC DRH - Manual Completo e Simplificado para Apresentacao

Estrutura deCodigo, Fluxo de Dados, Perguntas do Engenheiro e Negocio

1. OBJETIVO DO SISTEMA (EXPLICACAO SIMPLES)

O SERNIC DRH e uma plataforma digital desenvolvida para resolver o problema da gestao manual e desorganizada de pessoal no Servico Nacional de Investigacao Criminal de Mocambique. Antes do sistema, as informacoes dos funcionarios ficavam espalhadas em folhas de papel e ficheiros Excel soltos nas provincias. Com o SERNIC DRH, tudo passa a ser gerido num unico local seguro, permitindo emitir Guias de Marcha, Atos Administrativos, controlar a efetividade mensal e verificar a evolucao das carreiras em segundos.

2. ARQUITETURA DO SISTEMA (A ANALOGIA SIMPLIFICADA)

Para explicar a arquitetura a um engenheiro, o sistema esta dividido em 4 camadas principais que funcionam de forma semelhante a um restaurante:

- **React 19 (Frontend SPA - O Garcom e a Ementa):** E o ecrã com que o utilizador interage. Apresenta os formulários, tabelas e gráficos. Atualiza a tela instantaneamente sem dar "reload" na pagina.
- **Node.js (O Motor da Cozinha):** E o ambiente que executa o código JavaScript no servidor de forma assíncrona e rápida, processando múltiplos pedidos ao mesmo tempo.
- **Express v5 (O Chefe de Sala / Gestor de Pedidos):** Controla as rotas da API REST. Quando o frontend faz um pedido HTTP (ex: "Criar Funcionario"), o Express verifica a permissao do utilizador e chama a funcao correta.
- **SQLite / better-sqlite3 (A Despensa de Dados):** E a base de dados relacional onde todas as tabelas (funcionarios, utilizadores, despachos, faltas) ficam guardadas em disco com elevado desempenho e seguranca transaccional.

3. ESTRUTURA DAS PASTAS (ORGANIZACAO DO CODIGO)

Frontend (src/)

- **src/components/:** Conteúdo dos ecrãs divididos por módulos: employees/, adminActs/, access/, org/, vacations/.
- **src/components/common/:** Componentes reutilizáveis: DraggableModal (janelas flutuantes com Pointer Events nativos), DraggableTabs, ErrorBoundary.
- **src/hooks/:** Logica de dados isolada: useAdminActsData, useEmployees, useDraggable.
- **src/context/:** Guarda o estado global de autenticacao e sessao do utilizador.

Backend (server/)

- **server/index.js:** Ficheiro principal que liga o servidor Express, configura permissões de CORS e inicia o servidor.
- **server/db.js:** Cria e liga a base de dados SQLite, define o esquema de tabelas, índices e funções preparadas (DAO).
- **server/routes/:** Rotas REST divididas por recurso: employees.js, adminActs.js, auth.js, org.js.

4. FLUXO DE FUNCIONAMENTO (EXEMPLO: CADASTRO DE FUNCIONARIO)

- **Passo 1: Clique do Utilizador:** O operador preenche os dados do funcionario no formulario e clica no botao "Guardar".
- **Passo 2: Chamada da API REST:** O React executa a funcao `fetch("/api/employees", { method: "POST", body: JSON.stringify(dados) })`.
- **Passo 3: Recacao no Express:** O Express recebe o pedido na rota `server/routes/employees.js` e verifica a autenticacao.
- **Passo 4: Gravacao Segura na DB:** A rota chama a funcao de `server/db.js` que executa uma Prepared Statement: `db.prepare("INSERT INTO employees...").run(...)`.

- **Passo 5: Resposta e Atualizacao:** A base de dados confirma a gravacao, o Express responde HTTP 201 Created em JSON, e o React atualiza a tabela no ecrã sem dar reload.

5 A 8. FUNCIONAMENTO DE REQUISICOES, AUTENTICACAO E SEGURANCA

- **Metodos HTTP REST:** Usamos GET (buscar dados), POST (criar registos), PUT (atualizar) e DELETE (remover ou mover para a lixeira).
- **Autenticacao Segura:** As senhas sao encriptadas com a biblioteca BcryptJS (hash com sal). O sistema nunca guarda a senha em texto limpo.
- **Controlo por Perfis (RBAC):** O sistema suporta 4 niveis: Super Admin, Administrador, Operador e Leitor. Cada perfil so ve o que tem autorizacao.
- **Protecao SQL Injection:** Todas as consultas usam parametros vinculados (?). O SQLite trata as variaveis separadamente do comando SQL, impedindo injecoes de codigo.
- **Temporizador de Inatividade:** O SessionTimeoutModal.jsx monitoriza a atividade. Se o operador se afastar da mesa, a sessao e encerrada automaticamente.

9 A 12. MODULOS PRINCIPAIS, DESAFIOS E FUTURO

- **Modulos Notaveis:** Ficha Biografica, Organograma interativo em arvore, Atos Administrativos/Guias de Marcha em PDF, Mapa Mensal de Efetividade e Lixeira de Registos com auditoria.
- **Desafios de Engenharia:** Integracao perfeita com o Strict Mode do React 19 atraves de Hooks nativos com Pointer Events (sem pacotes legados instaveis) e geracao de PDFs institucionais formatados.
- **Evolucao Futura:** Possibilidade de migrar do SQLite para PostgreSQL sem alterar a interface do utilizador, e integracao com autenticacao governamental (SSO/Keycloak).

13. PERGUNTAS PROBABILISSIMAS DO ENGENHEIRO (COM RESPOSTAS PRONTAS)

Pergunta 1: Por que escolheste React e Node.js?

O que responder: "Escolhi React pela velocidade na criacao de interfaces SPA reativas sem reload. O Node.js da-nos um servidor REST leve e rapido com a mesma linguagem em todo o projeto."

Explicacao detalhada: O React gere o DOM virtual garantindo atualizacoes rapidas da UI. O Node.js lida com multiplos pedidos I/O nao bloqueantes de forma eficiente.

Pergunta 2: Como funciona o login e a seguranca da senha?

O que responder: "O utilizador envia NUI e senha. O backend valida a senha encriptada em BcryptJS e devolve a sessao com o perfil de acesso."

Explicacao detalhada: Utilizamos Bcrypt com salt para encriptar senhas na tabela users. O frontend armazena a sessao e o SessionTimeoutModal fecha a sessao em caso de inatividade.

Pergunta 3: Como garantes que nao ha SQL Injection?

O que responder: "Usamos a biblioteca better-sqlite3 exclusivamente com Prepared Statements e parametros vinculados (?, ?)."

Explicacao detalhada: Exemplo: db.prepare("SELECT * FROM employees WHERE id = ?").get(id). O motor da DB nunca interpreta as entradas do utilizador como comando SQL.

Pergunta 4: Como proteges os componentes e as rotas?

O que responder: "No Frontend usamos o **PermissionGuard** para bloquear ecras. No Backend, os **middlewares do Express** bloqueiam rotas nao autorizadas com **codigo HTTP 401 ou 403.**"

Explicacao detalhada: Mesmo que alguem tente fazer um pedido direto via Postman ou cURL para a API REST, o servidor recusa o pedido antes de consultar a base de dados.

Pergunta 5: Como tratas os erros na aplicacao?

O que responder: "No Frontend usamos o **ErrorBoundary do React** para capturar falhas da UI. No Backend usamos blocos **try...catch** em todas as rotas com respostas **JSON explicativas.**"

Explicacao detalhada: Se ocorrer um erro inesperado, a aplicacao nao vai abaixo. O utilizador ve um modal explicativo e o servidor devolve **HTTP 500** com registo no log de auditoria.

Pergunta 6: Como adicionarias um novo modulo sem alterar o codigo existente?

O que responder: "O codigo e **100% modular**. Criaria a tabela no **db.js**, a nova rota em **server/routes/**, a pasta do componente em **src/components/** e a nova aba no **Dashboard.**"

Explicacao detalhada: Nao e necessario alterar nenhuma linha dos modulos ja existentes. O principio de responsabilidade unica e modularidade e mantido rigorosamente.

Pergunta 7: O sistema segue algum padrao de arquitetura?

O que responder: "Sim, segue o padrao **Cliente-Servidor REST** com **Arquitetura em Camadas (Apresentacao em React, Servicos em Express e Dados em SQLite).**"

Explicacao detalhada: Esta separacao em camadas facilita a manutencao, testes e substituicao de componentes sem afetar as restantes partes do sistema.

Pergunta 8: Como farias a escalabilidade do sistema?

O que responder: "O frontend pode ser servido num **CDN/Nginx**, o backend **Node.js** containerizado em **Docker**, e o **SQLite** migrado para **PostgreSQL** se o volume nacional exigir."

Explicacao detalhada: Como a comunicacao e feita via API REST padronizada em JSON, a transicao da base de dados para PostgreSQL requer apenas a alteracao das queries no db.js.

14. DECISOES DE NEGOCIO E VALOR INSTITUCIONAL (SERNIC)

- **Centralizacao Total:** Eliminacao da dispersao de dados entre os orgaos centrais e as delegacoes provinciais.
- **Fim dos Ficheiros Excel Isolados:** Substituicao por uma base de dados relacional segura com copias de seguranca automaticas.
- **Rastreabilidade Imutavel (Audit Trail):** Registo de quem criou, editou ou removeu cada funcionario ou despacho.
- **Apoio a Decisao da Direcao:** Geracao instantanea de relatorios em PDF e mapas de efetividade para transferencias e promocoos.